

Maestría en Ciencia de Datos e Inteligencia Artificial

Curso: Programación con IA — Unidades 2 y 3

Actividad colaborativa · Fase I: Taller (actividad individual)

Programación orientada a objetos, decoradores y patrones de diseño asistidos por IA en Python

Caso de estudio: telemetría de sensores y gestión de incidentes en un laboratorio de electrónica

Estudiante: María Fernanda Rivera Sanclemente

Cédula: [completar]

Tutor(a): [completar]

Grupo: [completar]

Repositorio de código: <https://github.com/mrivera7-max/programacion-ia-udi>

Bucaramanga, Colombia

Septiembre de 2026

Contenido

1. Introducción

2. Desarrollo del taller

- 2.1 Ejercicio 1 — De un script estructurado a POO asistida por IA
- 2.2 Ejercicio 2 — Decorador de monitoreo para entrenamiento de modelos
- 2.3 Ejercicio 3 — Patrones Factory y Strategy para análisis de mensajes
- 2.4 Ejercicio 4 — Clasificación de prioridad con expresiones regulares
- 2.5 Aseguramiento de calidad del conjunto

3. Conclusiones

Referencias

Anexo A. Repositorio, reproducibilidad y estructura del código

1. Introducción

Los asistentes de inteligencia artificial generativa se han incorporado al ciclo de desarrollo de software como herramientas capaces de producir, en segundos, código sintácticamente correcto y plausible. Esa misma plausibilidad es su principal riesgo: el código generado suele funcionar en el caso típico y fallar en los bordes, donde se concentran los defectos que llegan a producción (Pearce et al., 2022). En consecuencia, el valor profesional del especialista se desplaza desde escribir cada línea hacia **especificar con precisión, verificar con evidencia y decidir la arquitectura**.

Este taller desarrolla los cuatro ejercicios de la Fase I sobre un caso de estudio coherente y cercano al perfil de la autora: la telemetría de sensores (temperatura, tensión y corriente) y los reportes de incidentes de un laboratorio de ingeniería electrónica. Sobre ese caso se aborda la refactorización de un proceso estructurado de carga y limpieza hacia Programación Orientada a Objetos (POO) con herencia y polimorfismo; la construcción de un decorador de monitoreo empaquetado como módulo independiente; la aplicación de los patrones *Factory* y *Strategy* (Gamma et al., 1994) en un sistema de análisis de mensajes; y un clasificador de prioridad basado en expresiones regulares, cuya elección entre IA simbólica y aprendizaje automático se sustenta empíricamente.

Metodológicamente, cada ejercicio sigue el mismo ciclo: **(1)** formulación del problema, **(2)** diseño de un *prompt* estructurado (rol, contexto, tarea, restricciones, formato y criterio de aceptación), **(3)** generación asistida, **(4)** verificación objetiva —pruebas automáticas con *pytest*, análisis estático con *ruff* y *mypy*, y comparación contra una línea base— y **(5)** refinamiento crítico. Todo el código se entrega comentado, con 17 pruebas automáticas que pasan en su totalidad, y es reproducible: los datos se generan con semilla fija (Python 3.12.3, pandas 3.0.2, scikit-learn 1.8.0). Las salidas mostradas en este documento son las obtenidas al ejecutar ese código.

El código completo, comentado y con integración continua está disponible públicamente en el repositorio del taller: <https://github.com/mrivera7-max/programacion-ia-udi> (ver Anexo A).

2. Desarrollo del taller

2.1 Ejercicio 1 — De un script estructurado a POO asistida por IA

a) Proceso original identificado

El proceso seleccionado es la carga y limpieza de la telemetría de cuatro sensores del laboratorio, exportada en cuatro formatos (.csv, .json, .txt tabulado y .xlsx). Los datos contienen, de forma controlada, los defectos típicos de la captura real: filas duplicadas, nulos, el texto «N/A» en campos numéricos, comas decimales, lecturas imposibles (999 °C) y categorías inconsistentes (« Ok», «falla»). La versión original resolvía todo en una única función con una cadena if/elif por formato:

```
def cargar_y_limpiar(ruta):
    # 1) Lectura: una rama por formato. Agregar un formato = editar esta función.
    if ruta.endswith(".csv"):
        df = pd.read_csv(ruta)
    elif ruta.endswith(".json"):
        with open(ruta, encoding="utf-8") as f:
            df = pd.DataFrame(json.load(f))
    elif ruta.endswith(".txt"):
        df = pd.read_csv(ruta, sep="\t")
    elif ruta.endswith(".xlsx"):
        df = pd.read_excel(ruta)
    else:
        print("Formato no soportado")
        return None

    # 2) Limpieza: reglas mezcladas con la lectura (sin separación de responsabilidades).
    df = df.drop_duplicates()
    for col in ["temperatura_c", "voltaje_v", "corriente_ma"]:
        df[col] = df[col].astype(str).str.replace(",", ".")
        df[col] = pd.to_numeric(df[col], errors="coerce")
    df["estado"] = df["estado"].str.strip().str.upper()
    df = df[(df["temperatura_c"] > -40) & (df["temperatura_c"] < 125)]
    df = df.dropna()
    df = df.reset_index(drop=True)
    return df
```

Figura 1. Script original (paradigma estructurado), línea base de comparación.

El diseño presenta tres problemas estructurales: la función tiene al menos dos razones para cambiar (leer y limpiar), lo que viola el Principio de Responsabilidad Única; agregar un formato exige modificar código ya probado, contrario al principio Abierto/Cerrado (Martin, 2017); y los errores se comunican con print y return None, de modo que el llamador puede continuar con datos inexistentes.

b) Prompt diseñado

El *prompt* se construyó con seis componentes explícitos. El rol fija el nivel de experticia esperado; el contexto entrega el esquema y el código real, evitando que la IA invente columnas; las restricciones convierten las buenas prácticas en requisitos verificables; y el último requisito —resultado idéntico al original— es el criterio de aceptación que después se prueba automáticamente.

Rol: Actúa como ingeniero de software senior especializado en Python 3.12, pandas y diseño orientado a objetos (SOLID).

Contexto: Tengo un script estructurado (``cargar_y_limpiar(ruta)``) que carga telemetría de sensores (columnas: `id_sensor`, `timestamp`, `temperatura_c`, `voltaje_v`, `corriente_ma`, `estado`) desde archivos `.csv`, `.json`, `.txt` (tabulado) y `.xlsx`, usando una cadena `if/elif` por extensión, y luego limpia los datos: elimina duplicados, convierte comas decimales a punto, fuerza tipos numéricos, normaliza ``estado`` (`strip + mayúsculas`), filtra temperatura fuera de `(-40, 125) °C` y elimina nulos. [Se adjuntó el código original completo].

Tarea: Refactoriza el script a Programación Orientada a Objetos:

1. Una clase base abstracta ``CargadorDatos`` (módulo ``abc``) que defina la interfaz común y la lógica de limpieza compartida.
2. Una subclase por formato (`CSV`, `JSON`, `TXT`, `XLSX`) que solo implemente la lectura, aprovechando herencia y polimorfismo.
3. Un mecanismo para obtener el cargador adecuado según la extensión.

Restricciones:

- Python 3.12, solo pandas y la biblioteca estándar.
- Tipado explícito (typing), docstrings estilo Google, PEP 8.
- No usar ``print`` para errores: usar excepciones propias y ``logging``.
- El resultado debe ser idéntico al del script original para los mismos datos.

Formato de salida: un único módulo Python y un bloque ``if __name__ == "__main__":`` que demuestre la carga polimórfica de los cuatro formatos.

Figura 2. Prompt enviado al asistente de IA.

c) Sugerencia inicial de la IA y validación

La primera respuesta del asistente se registró sin cambios (archivo `sugerencia_inicial_ia.py`). A simple vista cumple el pedido: clase base abstracta, cuatro subclases y un método de obtención. Los comentarios [AUDITORÍA] se agregaron después de validarla:

```
def limpiar(self, df: pd.DataFrame) -> pd.DataFrame:
    df = df.drop_duplicates()
    df = df.dropna() # [AUDITORÍA] ERROR 1: dropna ANTES de to_numeric.
    for col in ["temperatura_c", "voltaje_v", "corriente_ma"]:
        df[col] = df[col].astype(str).str.replace(",", ".")
        # [AUDITORÍA] "N/A" se vuelve NaN aquí, después del dropna -> nulos filtrados.
        df[col] = pd.to_numeric(df[col], errors="coerce")
    df["estado"] = df["estado"].str.strip().str.upper()
    df = df[(df["temperatura_c"] > -40) & (df["temperatura_c"] < 125)]
    return df.reset_index(drop=True)

def cargar(self) -> pd.DataFrame:
    try:
        return self.limpiar(self.leer())
    except Exception as e: # [AUDITORÍA] ERROR 2: silencia cualquier fallo
        print(f"Error: {e}")
        return pd.DataFrame() # y devuelve un DataFrame vacío "válido".
```

Figura 3. Fragmento de la sugerencia inicial de la IA con los hallazgos de auditoría.

La validación consistió en ejecutar la sugerencia sobre los cuatro formatos y compararla con la línea base. El resultado fue:

formato	filas	nulos	tipo timestamp	¿igual a base?
.csv	470	0	str	sí
.json	480	10	datetime64[us]	NO
.txt	470	0	str	sí
.xlsx	470	0	str	sí

Error: [Errno 2] No such file or directory: 'no_existe.csv'

Archivo inexistente -> la IA devuelve DataFrame vacío de forma silenciosa: `shape=(0, 0)`

Versión corregida -> lanza `ErrorCargaDatos`: No existe el archivo: `no_existe.csv`

Figura 4. Auditoría de la sugerencia inicial: el defecto solo se manifiesta con JSON.

d) Justificación técnica: análisis crítico de la limitación encontrada

Error principal: orden incorrecto de las operaciones de limpieza. La IA eliminó nulos (dropna) *antes* de convertir las columnas a numérico. El texto «N/A» no es nulo mientras es texto; se convierte en NaN después, con `to_numeric(errors='coerce')`, cuando el filtro ya se aplicó. Lo relevante para el análisis es *por qué* el defecto pasa desapercibido: `read_csv` interpreta «N/A» como nulo por defecto, de modo que con CSV, TXT y XLSX el resultado es correcto por casualidad. Solo el lector JSON conserva el texto literal y deja pasar 10 filas con nulos (480 filas en lugar de 470). Es un error **dependiente del formato**: una prueba solo con CSV —el caso típico que la IA optimiza— lo habría aprobado. El mismo lector JSON, además, usa `read_json`, que infiere `timestamp` como `datetime64`, de modo que las subclases no devuelven un esquema común y el polimorfismo se rompe en su promesa central: que el cliente no necesite saber qué subclase usa.

Errores secundarios. (i) El bloque `except Exception` imprime y devuelve un DataFrame vacío: un archivo inexistente produce `shape=(0, 0)` sin excepción, un fallo silencioso que en un *pipeline* de datos se propaga como un conjunto «válido» vacío. (ii) El `if/elif` del script original solo se trasladó a `obtener_cargador`; la IA cumplió la letra del pedido (usar clases) pero no su intención (extensibilidad). Esto ilustra un patrón general: el asistente reproduce la estructura del código que recibe como contexto, incluidos sus defectos de diseño.

e) Diseño corregido

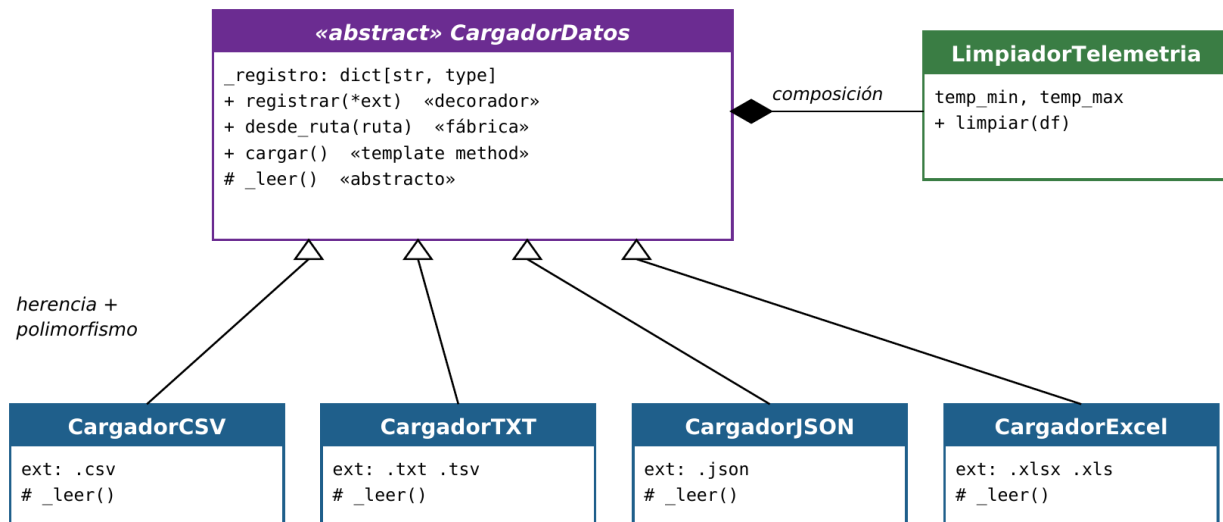


Figura 5. Diagrama de clases de la solución corregida.

La corrección aplica cuatro decisiones: **Template Method** en `cargar()`, que fija el flujo invariable (verificar existencia → leer → validar esquema → limpiar) y delega en cada subclase únicamente `_leer()`; **lectura como texto** (`dtype=str`) en todas las subclases, de modo que la conversión de tipos ocurre en un solo lugar y el esquema es idéntico sin importar el formato; **limpieza separada** en `LimpradorTelemetria`, inyectable y probable sin archivos; y **registro por decorador de clase**, que reemplaza la cadena condicional por un diccionario extensión → clase.

```
@classmethod
def registrar(cls, *extensiones: str):
    """Decorador de clase que asocia extensiones con una subclase."""
    def _decorar(subclase: type[CargadorDatos]) -> type[CargadorDatos]:
        for ext in extensiones:
            cls._registro[ext.lower()] = subclase
        return subclase
    return _decorar

@classmethod
def desde_ruta(cls, ruta: str | Path, **kwargs: object) -> CargadorDatos:
    """Fábrica: devuelve el cargador adecuado según la extensión del archivo.

    Raises:
        ErrorCargaDatos: Si la extensión no tiene cargador registrado.
    """
    ext = Path(ruta).suffix.lower()
    try:
        return cls._registro[ext](ruta, **kwargs) # type: ignore[arg-type]
    except KeyError:
        soportados = ", ".join(sorted(cls._registro))
        raise ErrorCargaDatos(f"Formato '{ext}' no soportado. Use: {soportados}") from None

# ----- flujo común
def cargar(self) -> pd.DataFrame:
    """Lee, valida y limpia el archivo. Los errores se propagan, no se ocultan.

    Raises:
        ErrorCargaDatos: Si el archivo no existe, no se puede leer o le
            faltan columnas.
    """
    if not self.ruta.exists():
        raise ErrorCargaDatos(f"No existe el archivo: {self.ruta}")
    try:
        crudo = self._leer()
    except (ValueError, OSError, json.JSONDecodeError) as exc:
        raise ErrorCargaDatos(f"No se pudo leer {self.ruta.name}: {exc}") from exc
    self._validar_esquema(crudo)
    limpio = self.limpiador.limpiar(crudo)
    logger.info("%s: %d filas crudas -> %d filas limpias",
                self.ruta.name, len(crudo), len(limpio))
    return limpio
```

Figura 6. Registro, fábrica y flujo común en la clase base (extracto de cargadores.py).

```
# ----- subclases
@CargadorDatos.registrar(".csv")
class CargadorCSV(CargadorDatos):
    """Archivos separados por comas."""

    def _leer(self) -> pd.DataFrame:
        return pd.read_csv(self.ruta, dtype=str, encoding="utf-8")
```

Figura 7. Una subclase completa: agregar un formato es escribir solo su lectura.

f) Validación de la versión corregida

```

1) Equivalencia P00 vs. script original (referencia: CSV original)
=====
.csv   OK  (470, 6)  (idéntico a la línea base)
.json  OK  (470, 6)  (idéntico a la línea base)
.txt   OK  (470, 6)  (idéntico a la línea base)
.xlsx  OK  (470, 6)  (idéntico a la línea base)

3) Eficiencia con 200 000 filas (mejor de 3 ejecuciones)
=====
Estructurado: 0.760 s | P00: 0.575 s | relación P00/estructurado = 0.76

4) Extensibilidad: nuevo formato .psv (separado por '|') sin editar el módulo
=====
CargadorPSV registrado -> (470, 6) OK

```

Figura 8. Equivalencia con la línea base, eficiencia y extensibilidad (validar_ej1.py).

Los cuatro formatos producen un DataFrame idéntico al del script original (`assert_frame_equal`). En eficiencia, con 200 000 filas, la versión POO tarda 0,58 s frente a 0,76 s. Conviene no atribuir esa mejora a la orientación a objetos: el despacho polimórfico ocurre una vez por archivo, no por fila, y su costo es despreciable. La ganancia proviene de leer como texto y convertir una sola vez, evitando la doble inferencia de tipos del original. La conclusión correcta es que el diseño modular **no penaliza el rendimiento**. Finalmente, el formato .psv se incorporó desde otro archivo, sin modificar una línea del módulo.

g) Mejora en escalabilidad y mantenimiento frente al paradigma estructurado

Tabla 1. Comparación del diseño estructurado y el diseño orientado a objetos.

Criterio	Script estructurado	Diseño POO corregido
Agregar un formato	Editar la función central y volver a probar todo el flujo.	Crear una subclase con <code>@registrar</code> ; el código existente no cambia (OCP).
Cambiar reglas de limpieza	Modificar la misma función que lee archivos.	Injectar otro <code>LimpiadorTelemetria</code> (p. ej., otros límites por tipo de sensor).
Pruebas	Solo pruebas de extremo a extremo con archivos reales.	El limpiador se prueba con DataFrames en memoria; cada lector, por separado.
Manejo de errores	<code>print + None</code> : fallos silenciosos.	Excepción de dominio <code>ErrorCargaDatos</code> con causa encadenada.
Consistencia de esquema	Depende del lector de cada rama.	Garantizada por el contrato de <code>_leer()</code> (todo como texto).
Trabajo en equipo	Conflictos de fusión en una sola función.	Cada integrante trabaja en su propia subclase o regla.

Con visión de futuro, esta arquitectura admite sin rediseño nuevas fuentes —Parquet, una API REST o un *broker* MQTT de sensores— como subclases adicionales, y permite paralelizar la carga de múltiples archivos porque cada cargador es independiente y sin estado compartido.

2.2 Ejercicio 2 — Decorador de monitoreo para entrenamiento de modelos

a) Organización en paquete

La herramienta se implementó como el paquete `monitoreo`, independiente del código de entrenamiento. Su `__init__.py` expone solo la API pública (`__all__`), de modo que los detalles internos pueden cambiar sin afectar a los consumidores:

```
monitoreo/
├── __init__.py      # API pública: monitorear, historial, RegistroEjecucion
├── decoradores.py   # implementación del decorador
└── ej2_entrenamiento.py # script principal: solo lógica de entrenamiento

# Uso en el script principal:
from monitoreo import historial, monitorear
```

Figura 9. Estructura del paquete e importación desde el script principal.

b) Implementación del decorador

```
log = logger or _logger

def decorador(f: Callable[..., Any]) -> Callable[..., Any]:
    @functools.wraps(f) # preserva __name__, __doc__, __wrapped__
    def envoltura(*args: Any, **kwargs: Any) -> Any:
        inicio = datetime.now()
        t0 = time.perf_counter() # reloj monotónico de alta resolución
        try:
            resultado = f(*args, **kwargs)
        except Exception as exc: # nunca BaseException (Ctrl+C, SystemExit)
            dur = time.perf_counter() - t0
            historial.append(RegistroEjecucion(f.__qualname__, inicio, dur, False,
                                              f"{type(exc).__name__}: {exc}", etiquetas))
            log.error("x %s falló tras %.3f s -> %s: %s",
                     f.__qualname__, dur, type(exc).__name__, exc,
                     exc_info=log.isEnabledFor(logging.DEBUG))
            if relanzar:
                raise
            return None
        dur = time.perf_counter() - t0
        historial.append(RegistroEjecucion(f.__qualname__, inicio, dur, True,
                                          etiquetas=etiquetas))
        nivel = logging.WARNING if umbral_s is not None and dur > umbral_s else logging.INFO
        log.log(nivel, "✓ %s completado en %.3f s%s", f.__qualname__, dur,
                " (supera umbral)" if nivel == logging.WARNING else "")
        return resultado
    return envoltura

# Permite @monitorear (func recibido directamente) y @monitorear(...).
return decorador(func) if callable(func) else decorador
```

Figura 10. Núcleo del decorador `@monitorear` (`decoradores.py`).

El ejercicio se apoya en que en Python las funciones son **objetos de primera clase** (Ramalho, 2022): `monitorear` recibe una función como argumento, define `envoltura` —una clausura que conserva acceso a `f`, `relanzar` y `umbral_s`— y la devuelve como valor. El decorador admite dos formas (`@monitorear` y `@monitorear(...)`) al detectar si su primer argumento es invocable. Las decisiones de diseño relevantes son:

- ``functools.wraps`` preserva nombre, *docstring* y `__wrapped__`; sin él, las trazas y la documentación mostrarían «envoltura» en lugar de la función real.
- ``time.perf_counter`` es monotónico y de alta resolución; `time.time` puede saltar por ajustes del reloj del sistema.

- Se captura `Exception`, no `BaseException`, para no interceptar `KeyboardInterrupt` ni `SystemExit`.
- `relanzar=True` por defecto (*fail-fast*): la opción que continúa tras un error debe elegirse explícitamente, nunca ocurrir por omisión —justo el defecto observado en la sugerencia de la IA del Ejercicio 1.
- Tipado con `ParamSpec` y sobrecargas: `mypy` conoce la firma exacta de la función decorada y sabe que solo puede devolver `None` cuando `relanzar=False`.

c) Script principal y resultados

```
@monitorear(fase="datos")
def preparar_datos(ruta: str) -> Datos:
    """Carga la telemetría (reutiliza el Ejercicio 1) y separa train/test."""
    df = CargadorDatos.desde_ruta(ruta).cargar()
    y = (df["estado"] == "FALLA").astype(int)
    return train_test_split(df[X_COLS], y, test_size=0.3, random_state=42, stratify=y)

@monitorear(umbral_s=0.25, fase="entrenamiento")
def entrenar(modelo: Pipeline, x: pd.DataFrame, y: pd.Series) -> Pipeline:
    """Ajusta el modelo recibido (función genérica: sirve para cualquier estimador)."""
    return modelo.fit(x, y)

@monitorear(fase="evaluacion")
def evaluar(modelo: Pipeline, x: pd.DataFrame, y: pd.Series) -> float:
    """Devuelve el F1 macro en el conjunto de prueba."""
    return f1_score(y, modelo.predict(x), average="macro")

@monitorear(relanzar=False, fase="entrenamiento")
def entrenar_con_error(x: pd.DataFrame, y: pd.Series) -> Pipeline:
    """Caso de prueba: hiperparámetro inválido -> la excepción se registra y el pipeline sigue."""
    return make_pipeline(StandardScaler(), LogisticRegression(C=-1.0)).fit(x, y)
```

Figura 11. Funciones de entrenamiento decoradas: no contienen código de monitoreo.

```
16:02:59 | INFO | telemetria.csv: 510 filas crudas -> 470 filas limpias
16:02:59 | INFO | ✓ preparar_datos completado en 0.010 s
16:02:59 | INFO | ✓ entrenar completado en 0.005 s
16:02:59 | INFO | ✓ evaluar completado en 0.003 s
-> Regresión logística: F1 macro = 0.663
16:03:00 | WARNING | ✓ entrenar completado en 0.299 s (supera umbral)
16:03:00 | INFO | ✓ evaluar completado en 0.019 s
-> Random Forest (300 árboles): F1 macro = 0.785
16:03:00 | ERROR | x entrenar_con_error falló tras 0.002 s -> InvalidParameterError: The 'C'
parameter of LogisticRegression must be a float in the range (0.0, inf]. Got -1.0 instead.
-> Modelo con error devuelto: None (el proceso no se detuvo)

Resumen del historial de monitoreo:
función      fase  duración_s  éxito  error
preparar_datos  datos    0.0099  True
entrenar  entrenamiento    0.0048  True
evaluar    evaluacion    0.0029  True
entrenar  entrenamiento    0.2985  True
evaluar    evaluacion    0.0187  True
entrenar_con_error  entrenamiento    0.0019 False InvalidParameterError: The 'C' parameter of L

Metadatos preservados por functools.wraps: nombre='entrenar', doc='Ajusta el modelo recibido
(función genérica: sirve para cualquier estimador).'
```

Figura 12. Ejecución de `ej2_entrenamiento.py`: tiempos, advertencia por umbral y error gestionado.

La salida evidencia las tres funciones del decorador: registra tiempos de cada etapa; emite una advertencia cuando el Random Forest supera el umbral de 0,25 s; y, ante un hiperparámetro inválido ($C=-1$), registra la excepción y permite que el proceso continúe porque esa función se decoró con `re lanzar=False`.

d) Justificación: SRP, DRY y desacoplamiento

Responsabilidad Única (SRP). Cada unidad tiene una sola razón para cambiar: entrenar cambia si cambia la forma de ajustar modelos; el decorador, si cambia la política de observabilidad. Ninguna función de entrenamiento contiene `try/except` ni cronómetros. **DRY** (Hunt y Thomas, 1999): el conocimiento sobre cómo medir y registrar tiene una única representación; se aplica a cuatro funciones sin duplicar código, y corregirlo o ampliarlo es un cambio en un solo lugar.

Por qué el desacoplamiento es fundamental. El monitoreo es una preocupación transversal: afecta a muchas funciones pero no pertenece a ninguna. Mezclarlo con la lógica de negocio produce código disperso y enredado, difícil de leer y de probar. Separado, se obtiene: (i) funciones de entrenamiento que se prueban sin efectos colaterales; (ii) la posibilidad de cambiar el destino de las métricas —hoy una lista en memoria, mañana MLflow o Prometheus— tocando solo el paquete; y (iii) una política uniforme para todo el equipo. En la práctica de MLOps, donde la trazabilidad de tiempos, fallos y metadatos de cada ejecución es requisito de reproducibilidad, esta separación es la base sobre la cual se construye la observabilidad de los modelos.

2.3 Ejercicio 3 — Patrones Factory y Strategy para análisis de mensajes

a) Aplicación de los patrones

El sistema debe analizar los mensajes del laboratorio con distintos algoritmos (prioridad por reglas, tono, un modelo de aprendizaje automático) y permitir elegirlos y agregarlos sin reescribir el sistema. Los dos patrones resuelven problemas complementarios (Gamma et al., 1994):

Tabla 2. Rol de cada patrón en el sistema de análisis de mensajes.

Patrón	Problema que resuelve	Aplicación en el sistema
Strategy (comportamiento)	Varios algoritmos intercambiables para la misma tarea.	EstrategiaAnálisis define el contrato analizar(msg) → Resultado; AnalizadorMensajes (contexto) delega en la estrategia actual y puede cambiarla en tiempo de ejecución.
Factory (creacional)	Crear objetos sin acoplar al cliente a las clases concretas.	FabricaEstrategias mantiene un registro nombre → constructor; la estrategia se elige por configuración (JSON/YAML), no por código.

```
class EstrategiaAnálisis(ABC):
    """Contrato común (interfaz Strategy)."""

    nombre: str = "base"

    @abstractmethod
    def analizar(self, mensaje: str) -> Resultado:
        """Analiza un mensaje y devuelve un ``Resultado``."""

class EstrategiaPrioridadReglas(EstrategiaAnálisis):
    """Prioridad por reglas regex (reutiliza el Ejercicio 4: DRY)."""

    nombre = "prioridad_reglas"

    def analizar(self, mensaje: str) -> Resultado:
        c = reglas_prioridad.clasificar(mensaje)
        return Resultado(self.nombre, c.prioridad.name.lower(), 1.0, ", ".join(c.evidencias))

# ... EstrategiaSentimientoLexico y EstrategiaModeloML siguen el mismo contrato ...

class AnalizadorMensajes:
    """Contexto del patrón Strategy: delega el análisis en la estrategia actual."""

    def __init__(self, estrategia: EstrategiaAnálisis) -> None:
        self._estrategia = estrategia

    @property
    def estrategia(self) -> EstrategiaAnálisis:
        return self._estrategia

    @estrategia.setter
    def estrategia(self, nueva: EstrategiaAnálisis) -> None:
        """Permite cambiar el algoritmo en tiempo de ejecución."""
        self._estrategia = nueva

    def analizar_lote(self, mensajes: Iterable[str]) -> list[Resultado]:
        return [self._estrategia.analizar(m) for m in mensajes]
```

Figura 13. Interfaz Strategy, una estrategia concreta y el contexto (estrategias.py).

```

class FabricaEstrategias:
    """Registro nombre -> constructor. Nuevas estrategias no modifican la fábrica."""

    _registro: dict[str, Constructor] = {}

    @classmethod
    def registrar(cls, nombre: str, constructor: Constructor) -> None:
        if nombre in cls._registro:
            raise ValueError(f"La estrategia '{nombre}' ya está registrada")
        cls._registro[nombre] = constructor

    @classmethod
    def crear(cls, nombre: str, **parametros: Any) -> EstrategiaAnálisis:
        """Instancia la estrategia pedida.

        Raises:
            KeyError: Si el nombre no está registrado (mensaje con las opciones).
        """
        try:
            return cls._registro[nombre](**parametros)
        except KeyError:
            raise KeyError(f"Estrategia '{nombre}' desconocida. "
                           f"Disponibles: {sorted(cls._registro)}") from None

    @classmethod
    def desde_config(cls, config: dict[str, Any]) -> EstrategiaAnálisis:
        """Crea la estrategia desde un dict (p. ej., leído de JSON/YAML)."""
        return cls.crear(config["estrategia"], **config.get("parametros", {}))

FabricaEstrategias.registrar("prioridad_reglas", EstrategiaPrioridadReglas)
FabricaEstrategias.registrar("sentimiento", EstrategiaSentimientoLexico)
FabricaEstrategias.registrar("prioridad_ml", EstrategiaModeloML)

```

Figura 14. Fábrica con registro (fabrica.py).

```

Estrategias registradas: ['prioridad_ml', 'prioridad_reglas', 'sentimiento']

```

estrategia	mensaje	etiqueta	confianza
prioridad_reglas	Falla crítica en la fuente de alimentaci	alta	1.00
prioridad_reglas	El proyector del aula 101 no enciende	media	1.00
prioridad_reglas	Gracias por el apoyo en la práctica de a	baja	1.00
sentimiento	Falla crítica en la fuente de alimentaci	negativo	1.00
sentimiento	El proyector del aula 101 no enciende	negativo	1.00
sentimiento	Gracias por el apoyo en la práctica de a	positivo	1.00
prioridad_ml	Falla crítica en la fuente de alimentaci	alta	0.57
prioridad_ml	El proyector del aula 101 no enciende	media	0.48
prioridad_ml	Gracias por el apoyo en la práctica de a	baja	0.50

```

Nueva estrategia 'longitud': [('extenso', '10 palabras'), ('breve', '7 palabras'), ('extenso', '9 palabras')]
Validación de la fábrica: "Estrategia 'inexistente' desconocida. Disponibles: ['longitud', 'prioridad_ml', 'prioridad_reglas', 'sentimiento']"

```

Figura 15. Ejecución de demo_ej3.py: tres estrategias sobre los mismos mensajes y extensión en caliente.

La combinación aporta flexibilidad real: la estrategia se selecciona desde un diccionario de configuración, se sustituye en ejecución sin modificar el analizador y una estrategia nueva (longitud) se incorpora registrándola, sin tocar la fábrica ni el contexto. Además, la estrategia de reglas reutiliza el clasificador del Ejercicio 4 en lugar de duplicarlo.

b) Ejemplo de uso de una herramienta de IA generativa

Para agregar una estrategia se usaría un *prompt* que entregue el contrato existente como restricción, de modo que la IA genere una pieza que encaje en el sistema en lugar de uno nuevo:

```

Rol: ingeniero de software Python 3.12 experto en patrones de diseño.
Contexto: [se adjunta estrategias.py]. Toda estrategia hereda de EstrategiaAnalisis
e implementa analizar(mensaje: str) -> Resultado.
Tarea: implementa EstrategiaEntidades, que detecte equipos (osciloscopio, multímetro,
fuente, servidor...) y ubicaciones ('aula 204', 'mesa 5') con expresiones regulares.
Restricciones: solo biblioteca estándar; no modifiques clases existentes; typing,
docstrings y PEP 8; texto normalizado con ej4_prioridad.normalizar.
Formato: la clase, su línea de registro en la fábrica y 5 pruebas pytest con casos
borde (mensaje vacío, tildes, mayúsculas, sin entidades).
    
```

Figura 16. Prompt para generar una nueva estrategia compatible con el sistema.

Sobre el código obtenido se aplicarían validaciones en cinco niveles, varias ya automatizadas en este taller:

Tabla 3. Validaciones sobre código generado por IA.

Nivel	Validación	Herramienta / evidencia
Contrato	Hereda de la interfaz, devuelve Resultado, no modifica clases existentes.	<i>mypy</i> (0 errores) y revisión del <i>diff</i> .
Funcional	Casos típicos y de borde, incluidos los que el prompt no mencionó.	<i>pytest</i> : 17/17 pruebas del taller.
Estilo y defectos	PEP 8, importaciones no usadas, errores comunes.	<i>ruff</i> : sin hallazgos.
Seguridad	Sin <code>eval/exec</code> , sin deserializar datos no confiables; verificar que los paquetes sugeridos existan (la IA puede inventar nombres).	Revisión manual y verificación en PyPI.
Validez estadística	Si la estrategia usa ML, evaluar fuera de la muestra de entrenamiento.	Validación cruzada (ver Ejercicio 4).

El último nivel no es teórico: en la salida anterior, las confianzas de `prioridad_ml` se calcularon sobre mensajes que el modelo vio durante su entrenamiento, por lo que son optimistas. Su desempeño real se mide en el Ejercicio 4.

2.4 Ejercicio 4 — Clasificación de prioridad con expresiones regulares

a) Diseño del script

`ej4_prioridad.py` lee todos los `.txt` de una carpeta (un mensaje por línea), clasifica cada mensaje y exporta un CSV. Decisiones principales: el texto se **normaliza** (minúsculas, sin tildes) para que «Crítica» y «critica» coincidan; los patrones usan **límites de palabra** (`\b`) para evitar falsos positivos —«fallecimiento» no activa «falla»—; las reglas son **datos declarativos** jerarquizados (ALTA se evalúa antes que MEDIA); una regla **numérica** captura lo que una lista de palabras no puede (`batería ≤ 15 %`); la lectura es perezosa (generador) y tolera archivos mal codificados; y cada clasificación devuelve las **evidencias** que la originaron.

```

# Reglas declarativas: el conocimiento experto vive en DATOS, no en if/elif.
# Los patrones se aplican sobre texto normalizado (minúsculas y sin tildes).
REGLAS: dict[Prioridad, list[str]] = {
    Prioridad.ALTA: [
        r"\burgente\b", r"\bcritic[oa]s?\b", r"\bincendio\b", r"\bhumo\b",
        r"\bchispas?\b", r"\bquemad[oa]\b", r"\bcortocircuito\b",
        r"\bdescarga electrica\b", r"\bpeligro\b", r"\briesgo electrico\b",
        r"\bca[ií]d[oa]\b", r"\bno responde\b", r"\bperdida de datos\b",
        r"\baccesso no autorizado\b", r"\bde inmediato\b", r"\bpelad[oa]\b",
    ],
    Prioridad.MEDIA: [
        r"\bfall[aoó]w*\b", r"\berrorw*\b", r"\brevisar\b", r"\blent[oa]\b",
        r"\bno enciende\b", r"\bintermitente\b", r"\bretraso\b",
        r"\bmantenimiento\b", r"\bactualizar\b", r"\binestables?\b",
    ],
}

# Regla numérica (algo que una lista de palabras no captura): batería <= 15 %.
PATRON_BATERIA = re.compile(r"\bbateria\b.*?\b(\d{1,3})\s*%")
UMBRAL_BATERIA = 15

_COMPILADAS = {p: [re.compile(x) for x in pats] for p, pats in REGLAS.items()}

def normalizar(texto: str) -> str:
    """Minúsculas y sin tildes, para que 'Crítica' y 'critica' coincidan."""
    nfkd = unicodedata.normalize("NFKD", texto.lower())
    return "".join(c for c in nfkd if not unicodedata.combining(c))

@dataclass(frozen=True)
class Clasificacion:
    """Resultado explicable: prioridad + evidencia que la justifica."""

    mensaje: str
    prioridad: Prioridad
    evidencias: tuple[str, ...]

def clasificar(mensaje: str) -> Clasificacion:
    """Asigna la prioridad más alta cuyas reglas se activen (BAJA por defecto).

    Args:
        mensaje: Texto del mensaje original.

    Returns:
        Clasificación con las coincidencias que la originaron (trazabilidad).
    """
    texto = normalizar(mensaje)
    bat = PATRON_BATERIA.search(texto)
    if bat and int(bat.group(1)) <= UMBRAL_BATERIA:
        return Clasificacion(mensaje, Prioridad.ALTA, (f"bateria {bat.group(1)}%",))
    for nivel in (Prioridad.ALTA, Prioridad.MEDIA): # de mayor a menor
        hallazgos = tuple(m.group(0) for rx in _COMPILADAS[nivel] if (m := rx.search(texto)))
        if hallazgos:
            return Clasificacion(mensaje, nivel, hallazgos)
    return Clasificacion(mensaje, Prioridad.BAJA, ())

```

Figura 17. Reglas y función de clasificación (extracto de ej4_prioridad.py).

b) Resultados

mensaje	prioridad	evidencias
URGENTE: el osciloscopio del laboratorio 3 gener	alta	urgente, chispas
Falla crítica en la fuente de alimentación, huel	alta	critica, quemado
El servidor de simulación está caído y no respon	alta	caido, no responde
Por favor revisar el multímetro 12, marca lectur	media	revisar, inestables
Se requiere actualizar MATLAB en las estaciones	media	actualizar
Gracias por el apoyo en la práctica de ayer	baja	
Recordatorio: reunión de docentes el viernes	baja	
El generador de funciones presenta un error inte	media	error, intermitente
Cortocircuito en la protoboard de la mesa 5, rie	alta	cortocircuito, riesgo electrico
Solicito préstamo de cables banana para el marte	baja	
... (20 filas más)		

Distribución: {'alta': 11, 'media': 10, 'baja': 9} -> datos/resultados_prioridad.csv

Reglas (IA simbólica), n=30: exactitud = 96.7%

	pred_alta	pred_media	pred_baja
real_alta	11	0	0
real_media	0	9	0
real_baja	0	1	9

Error: 'Pendiente revisar el inventario de resistencias' real=baja pred=media

Modelo scikit-learn (TF-IDF + LogReg, CV 5 pliegues, n=30): exactitud = 56.7%

Figura 18. Clasificación, matriz de confusión de las reglas y comparación con scikit-learn.

c) Elección: IA simbólica o scikit-learn

Se elige IA simbólica (sistema basado en reglas), por cuatro razones sustentadas en el contexto del problema y en evidencia:

- **Disponibilidad de datos.** Solo existen 30 mensajes etiquetados. Con validación cruzada estratificada de 5 pliegues, TF-IDF + regresión logística alcanza 56,7 % de exactitud, frente a 96,7 % de las reglas. Un modelo supervisado necesita muchos más ejemplos por clase para generalizar el vocabulario.
- **Conocimiento experto explícito.** En un laboratorio, «cortocircuito», «humo» o «descarga eléctrica» son alta prioridad por definición de seguridad, no por correlación estadística. Una regla codifica ese conocimiento de forma directa y auditable.
- **Explicabilidad.** Cada decisión viene con su evidencia (p. ej., «cortocircuito, riesgo electrico»). En decisiones con consecuencias —atender primero un riesgo eléctrico— se prefieren modelos interpretables (Rudin, 2019).
- **Costo y determinismo.** No requiere entrenamiento ni reentrenamiento, responde en microsegundos y siempre produce la misma salida para la misma entrada.

Limitaciones reconocidas. La exactitud de 96,7 % es optimista: las reglas se escribieron conociendo esos mismos mensajes, por lo que no es una estimación fuera de muestra. Las reglas tampoco entienden contexto: «Pendiente revisar el inventario» se clasificó como media por la palabra «revisar», y «Los errores de ayer ya se corrigieron» también activaría una regla (caso documentado en las pruebas). Mantener las reglas exige trabajo manual a medida que cambia el vocabulario.

Visión a futuro. La decisión no es permanente sino dependiente de la etapa. El camino recomendado es híbrido: usar las reglas como funciones de etiquetado débil para generar un corpus inicial (Ratner et al., 2020), validarlo manualmente y, cuando existan varios cientos de ejemplos por clase, entrenar un modelo de scikit-learn —o evaluar un modelo de lenguaje— que capture contexto y negación, manteniendo las reglas críticas de seguridad como una capa que el modelo no puede anular. Gracias al patrón Strategy del Ejercicio 3, ese cambio consiste en registrar una estrategia nueva, no en reescribir el sistema.

2.5 Aseguramiento de calidad del conjunto

Como cierre, todo el código se sometió a verificación automática: 17 pruebas unitarias y de integración, análisis de estilo y defectos (PEP 8 y reglas de *bugbear*, *naming* y modernización) y verificación estática de tipos. Estas mismas comprobaciones se ejecutan en el repositorio mediante integración continua (GitHub Actions) en cada cambio:

```
$ python -m pytest -q tests
..... [100%]
17 passed in 0.40s
$ ruff check .
All checks passed!
$ mypy (módulos del taller)
Success: no issues found in 9 source files
```

Figura 19. Pruebas (pytest), estilo (ruff) y tipado (mypy).

3. Conclusiones

La IA acelera la producción, pero la calidad depende de la verificación. La sugerencia inicial del Ejercicio 1 parecía correcta y funcionaba con tres de cuatro formatos; su defecto solo apareció al comparar contra una línea base con datos que incluían el caso borde. La lección metodológica es concreta: todo código generado debe someterse a un criterio de aceptación verificable —pruebas, equivalencia y análisis estático— definido *antes* de generarlo.

Los principios de diseño son el lenguaje para dirigir a la IA. Pedir «usa clases» produjo clases; pedir extensibilidad sin modificar código existente habría requerido nombrar el principio Abierto/Cerrado. Conocer SOLID, DRY y los patrones de diseño permite escribir *prompts* precisos y reconocer cuándo una respuesta cumple la forma pero no la intención.

La elección de la técnica de IA es una decisión de ingeniería basada en evidencia. El Ejercicio 4 mostró que, con pocos datos y conocimiento experto claro, un sistema de reglas supera a un modelo estadístico, y que esa conclusión debe revisarse cuando cambian las condiciones.

Aplicación desde el perfil profesional. Como ingeniera electrónica y docente del programa de Ingeniería Electrónica, estos contenidos tienen aplicación directa en tres frentes. En **investigación**, el decorador de monitoreo y la carga polimórfica son base para los experimentos de reconocimiento de Lengua de Señas Colombiana, donde conviene registrar de forma uniforme el tiempo y los fallos de cada entrenamiento, y donde el patrón Strategy permite comparar arquitecturas de reconocimiento intercambiándolas por configuración. En **ingeniería**, la arquitectura de cargadores es aplicable a la telemetría de sistemas embebidos e IoT, donde conviven múltiples formatos y protocolos. En **docencia**, el ciclo *prompt* → verificación → refinamiento es un modelo para enseñar a los estudiantes a usar la IA como asistente crítico y no como sustituto del criterio técnico, competencia que será central en la formación de ingenieros en los próximos años.

Referencias

- Castaño Giraldo, S. A. [Control Automático Educación]. (2022, 21 de diciembre). *Programación Orientada a Objetos en Python [Introducción] # 028* [Video]. YouTube. <https://www.youtube.com/watch?v=Gzjb6LeVZcU>
- Einarr07. (s. f.). *curso-poo-python* [Repositorio de código fuente]. GitHub. Recuperado el 25 de agosto de 2026, de <https://github.com/Einarr07/curso-poo-python>
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Hunt, A., y Thomas, D. (1999). *The pragmatic programmer: From journeyman to master*. Addison-Wesley.
- JimShapedCoding. (2021, 13 de octubre). *Object oriented programming with Python – Full course for beginners* [Video]. YouTube / freeCodeCamp.org. https://www.youtube.com/watch?v=Ej_02ICOlgs
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt y J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., y Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. En *2022 IEEE Symposium on Security and Privacy (SP)* (pp. 754–768). IEEE. <https://doi.org/10.1109/SP46214.2022.9833571>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

- Ramalho, L. (2022). *Fluent Python: Clear, concise, and effective programming* (2.ª ed.). O'Reilly Media.
- Ratner, A., Bach, S. H., Ehrenberg, H., Fries, J., Wu, S., y Ré, C. (2020). Snorkel: Rapid training data creation with weak supervision. *The VLDB Journal*, 29(2), 709–730. <https://doi.org/10.1007/s00778-019-00552-1>
- Rivera Sanclemente, M. F. (2026). *Taller Fase I – Programación con IA: POO, decoradores y patrones de diseño* [Repositorio de código fuente]. GitHub. <https://github.com/mrivera7-max/programacion-ia-udi>
- Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5), 206–215. <https://doi.org/10.1038/s42256-019-0048-x>
- Universidad de Investigación y Desarrollo. (s. f.). *Programación con IA: Unidades 2 y 3* [Material institucional del curso]. UDI Virtual.
- van Rossum, G., Warsaw, B., y Coghlan, N. (2001). *PEP 8 – Style guide for Python code*. Python Software Foundation. <https://peps.python.org/pep-0008/>

Anexo A. Repositorio, reproducibilidad y estructura del código

Repositorio público. El código completo y comentado, los datos, las pruebas automáticas y este informe están disponibles en:

<https://github.com/mrivera7-max/programacion-ia-udi>

El repositorio incluye licencia MIT, archivo `requirements.txt`, archivo de citación (`CITATION.cff`) y un flujo de integración continua (GitHub Actions) que ejecuta las pruebas, `ruff` y `mypy` en cada cambio, de modo que cualquier lector puede verificar que el código del taller funciona sin ejecutarlo localmente.

Reproducción local. Los datos son sintéticos y se generan con semilla fija (`numpy.random.default_rng(42)`); el entorno de referencia es Python 3.12.3, pandas 3.0.2 y scikit-learn 1.8.0.

```
git clone https://github.com/mrivera7-max/programacion-ia-udi
cd programacion-ia-udi
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt

python generar_datos.py # 1. genera datos/ (telemetría + mensajes)
python ej1_poo/procedural_original.py # Ej. 1: línea base estructurada
python ej1_poo/sugerencia_inicial_ia.py # Ej. 1: respuesta inicial de la IA (auditada)
python ej1_poo/cargadores.py # Ej. 1: versión P00 corregida
python ej1_poo/validar_ej1.py # Ej. 1: equivalencia, eficiencia, extensibilidad
python ej2_entrenamiento.py # Ej. 2: decorador @monitorear en entrenamiento
python ej3_mensajes/demo_ej3.py # Ej. 3: Factory + Strategy
python ej4_prioridad.py datos/mensajes --etiquetas datos/mensajes_etiquetados.csv --comparar-ml
python -m pytest -q tests && ruff check . && mypy ej1_poo/cargadores.py monitoreo ej3_mensajes
ej4_prioridad.py ej2_entrenamiento.py

# Calidad del código
python -m pytest -q tests
ruff check .
mypy ej1_poo/cargadores.py monitoreo ej3_mensajes ej4_prioridad.py ej2_entrenamiento.py
```

Figura 20. Comandos de instalación y ejecución.

ej1_poo/	Ej. 1: original estructurado, prompt, sugerencia de la IA, versión P00 corregida y script de validación
monitoreo/	Ej. 2: paquete con el decorador @monitorear
ej2_entrenamiento.py	Ej. 2: script principal (entrenamiento monitoreado)
ej3_mensajes/	Ej. 3: estrategias (Strategy) y fábrica (Factory)
ej4_prioridad.py	Ej. 4: clasificador de prioridad por expresiones regulares
generar_datos.py	Generación reproducible de los datos (semilla fija)
tests/	17 pruebas automáticas (pytest)
salidas/	Salidas de ejecución citadas en el informe
docs/	Informe del taller
.github/workflows/	Integración continua: pruebas, ruff y mypy

Figura 21. Estructura de carpetas del repositorio.

Declaración de uso de IA. En este taller se utilizó un asistente de IA generativa (Claude, de Anthropic) como herramienta de apoyo para generar propuestas de código, conforme a lo solicitado en los ejercicios. Todo el código fue ejecutado, probado y auditado; los hallazgos, las correcciones y las decisiones de diseño se documentan en cada ejercicio.